

CMPT 280

Topic 3: Timing Analysis

Mark G. Eramian

University of Saskatchewan

References

- Textbook, Chapter 3

Timing Analysis

Express the time required by an algorithm for an input of size n in Big-Oh notation. Also referred to as find the *time complexity* of an algorithm.

1. Determine time required as a function of the input size n (may use best-case or worst-case analysis).
 - Statement counting
 - Active operation
2. Simplify the resulting function to Big-Oh notation.
 - By inspection
 - By formal proof
3. If best and worst case analysis yield same result, may write as Big-Theta.

Exercise 1

Consider this method which returns true if the array `a` contains the string `s`.

```
1 public static bool contains(String[] a, String s)
2 {
3     int count = 0;
4     int i = 0;
5     while (i < a.length && !s.equals(a[i]))
6     {
7         i = i + 1;
8     }
9     return i < a.length;
10 }
```

- a) For an input of size n , what is the best case input?
- b) For an input of size n , what is the worst case input?
- c) Exactly how many statements are executed in the worst case? Best case?

Exercise 2

```
1 public static <T extends Comparable<T>>
2     void insertionSort(T[ ] a)
3 {
4     // For each string in the array...
5     int i = 1;
6     while (i < a.length)
7     {
8         // Get the i-th element.
9         T temp = a[i];
10
11        // Examine all of the elements that come before it, starting at the rightmost.
12        // If 'temp' comes before a[j], move a[j] one index to the right and continue.
13
14        int j = i - 1;
15        while (j >= 0 && temp.compareTo(a[j]) < 0)
16        {
17            a[j+1] = a[j];
18            j = j - 1;
19        }
20
21        // We've found the insertion point. Copy the string here and advance to the next insertion.
22        a[j+1] = temp;
23        i = i + 1;
24    }
25 }
```

- Exactly how many statements are executed in the worst case?

Exercise 3

- Repeat exercise 1 and 2 using the active operation approach. Exactly how many times does the active operation get executed?
- Remember: best case and worst case are not the smallest inputs. Think of the best/worst case for an input of size n .

Exercise 4

What is the Big-Oh membership of each of the following functions? We would like the tightest upper bound. Simplify by inspection.

- n
- $47n \log n + 10000n$
- $100n + 500 \log n + 1000$
- $\log n + 100\sqrt{n} + 76$
- n^2
- 5
- $2^n + n + \log n$
- $T_{contains}^B(n) = 4$
- $T_{contains}^W(n) = 2n + 4$
- $T_{insort}^W(n) = 1.5n^2 + 4.5n - 4$

A More Elaborate Example

Suppose method $q()$ calls another method `int k(int i)` with $T_k(m) = O(\log(m))$ for some m independent of i .

```
1 public void q()  
2 {  
3     int c = 0;  
4     for (int i = 1; i < n + 1; i++)  
5         c = c + k(i);  
6 }
```

Determine the time complexity (i.e. the time execution time as a function of input size expressed in Big-Oh notation) of this method using the active operation approach.

- What is the active operation?
- How many times does it execute?
- What is the cost of the active operation?
- What is the total amount of time required expressed in Big-Oh notation.

Exercise 5

Again, assume method k requires $O(\log m)$ time, m independent of i and n .

```
1 public void s()  
2 {  
3     int x = 0;  
4     for (int i = 0; i < n + 1; i++)  
5         x = x + k(i)*k(2*n - i);  
6     System.out.println(x);  
7 }
```

- What is the active operation?
- How many times does it execute?
- What is the cost of the active operation?
- What is the total amount of time required expressed in Big-Oh notation.

Exercise 6

```
1 static public int f(int n)
2 {
3     int result = 0;
4     double p = n;
5     while (p > 1)
6     {
7         p = p/2;
8         result = result + 1;
9     }
10    return result;
11 }
```

What is the time complexity of the method?

Combining Growth Functions

- Since $O(f(n))$ and $O(g(n))$ are sets, we can use these set properties and the definition of Big-Oh to show:

$$\begin{aligned}O(f(n)) + O(g(n)) &= O(f(n) + g(n)) \\&= O(2 \cdot \max(f(n), g(n))) \\&= O(\max(f(n), g(n)))\end{aligned}$$

$$\begin{aligned}O(f(n)) \cdot O(g(n)) &= O(f(n) \cdot g(n)) \\kO(f(n)) &= O(kf(n)) = O(f(n)) \text{ for any constant } k.\end{aligned}$$

- Same holds for Big-Theta. We will need at least one of these "rules" to solve our next problem.

Exercise 7

```
1 public void r()  
2 {  
3     q();  
4     for (int i = 1; i < m; i = 2*i)  
5         p();  
6     s();  
7 }
```

- Assuming that we know that:
 - $T_q(n, m) \in O(n \log m)$
 - $T_p(n) \in O(n^2)$
 - $T_s(m) \in O(m \log m)$

What is the time complexity of the method `r`? Use the active operation approach. There are three candidate active operations, what are they? Determine time required by all three; the one that requires the most time is reflective of the overall time complexity.

Exercise 8

- Now we have the tools to return to our list class and consider the time complexity of our list operations.
- Determine the worst-case time complexity of all of the methods in the LinkedList class.
- Are there any methods where the time complexity could be made better?

Do Growth Rates Matter?

- Does the growth rate make a difference, or are computers so fast that any algorithm can be done quickly?
- Some algorithms take huge amounts of time on even small problems.
- Linear vs. quadratic alone is big difference.

Do Growth Rates Matter?

Time to execute $f(n)$ operations at 10^6 operations per second.

n	$\log_2 n$	n	$n \log_2 n$	n^2	2^n
10	$3.3 \times 10^{-6}s$	$1.0 \times 10^{-5}s$	$3.3 \times 10^{-5}s$	$1.0 \times 10^{-4}s$	$1.0 \times 10^{-3}s$
100	$6.6 \times 10^{-6}s$	$1.0 \times 10^{-4}s$	$6.6 \times 10^{-4}s$	$1.0 \times 10^{-2}s$	$4.0 \times 10^{16}y^*$
1000	$9.97 \times 10^{-6}s$	$1.0 \times 10^{-3}s$	$9.97 \times 10^{-3}s$	$1.0s$	
10^5	$1.66 \times 10^{-5}s$	$0.1s$	$1.66s$	$2.78h$	
10^6	$1.99 \times 10^{-5}s$	$1s$	$19.9s$	$11.57d$	
10^7	$2.33 \times 10^{-5}s$	$10s$	$233s$	$3.17y$	

Do Growth Rates Matter?

Time to execute $f(n)$ operations at 10^6 operations per second.

n	$\log_2 n$	n	$n \log_2 n$	n^2	2^n
10	$3.3 \times 10^{-6}s$	$1.0 \times 10^{-5}s$	$3.3 \times 10^{-5}s$	$1.0 \times 10^{-4}s$	$1.0 \times 10^{-3}s$
100	$6.6 \times 10^{-6}s$	$1.0 \times 10^{-4}s$	$6.6 \times 10^{-4}s$	$1.0 \times 10^{-2}s$	$4.0 \times 10^{16}y^*$
1000	$9.97 \times 10^{-6}s$	$1.0 \times 10^{-3}s$	$9.97 \times 10^{-3}s$	$1.0s$	Why bother?
10^5	$1.66 \times 10^{-5}s$	$0.1s$	$1.66s$	$2.78h$	
10^6	$1.99 \times 10^{-5}s$	$1s$	$19.9s$	$11.57d$	
10^7	$2.33 \times 10^{-5}s$	$10s$	$233s$	$3.17y$	

* Est. age of universe: 1.3×10^{10} years.



Next Class

- **Next class reading:** Chapter 4: Iteration and Cursors